

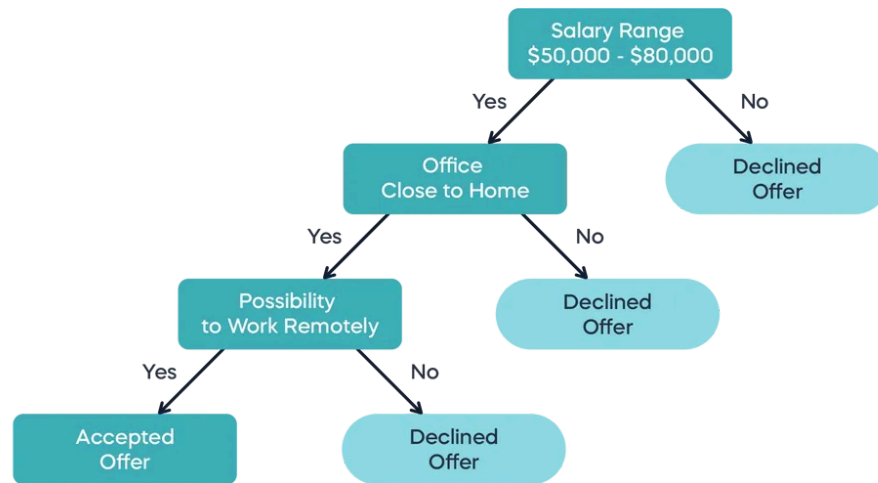
# SUPERVISED MACHINE LEARNING: CLASSIFICATION

## MODULE 4: DECISION TREES

### TABLE OF CONTENTS

|                                     |   |
|-------------------------------------|---|
| Overview.....                       | 2 |
| How They Work.....                  | 2 |
| Splitting Criteria.....             | 3 |
| Overfitting and Regularization..... | 5 |
| Pros and Cons.....                  | 6 |
| Implementation in Python.....       | 6 |
| Comparison with Other Models.....   | 7 |
| Key Takeaways.....                  | 7 |

## Overview



- **Decision Trees** are supervised learning algorithms used for **classification** and **regression** tasks.
- They work by **recursively splitting** data into subsets based on feature values until a stopping criterion is met.
- Decision trees are **non-parametric** and make **no statistical assumptions** about data.

## How They Work

- Each **node (nút)** represents a question about an attribute.
- Each **branch (nhánh)** corresponds to an answer (True/False).
- Each **leaf (lá)** gives the final prediction (class label or value).

### **Example (classification):**

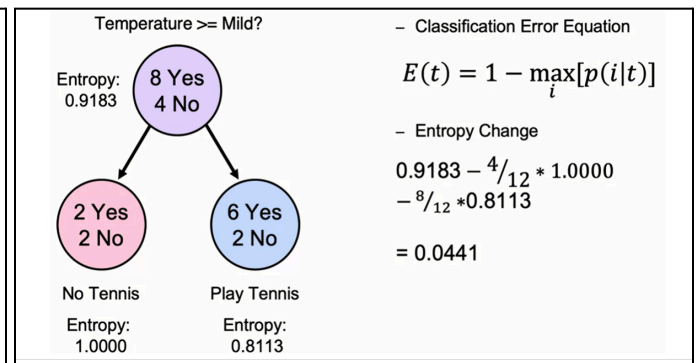
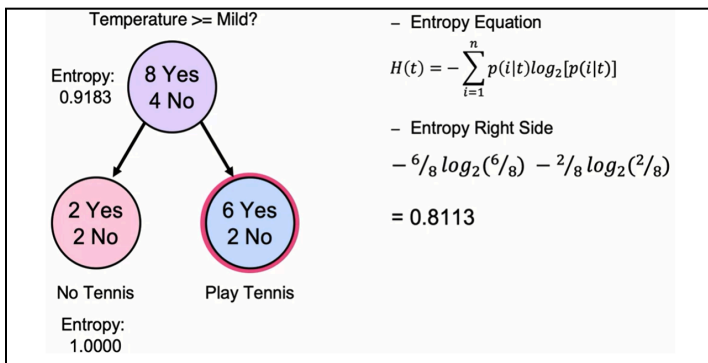
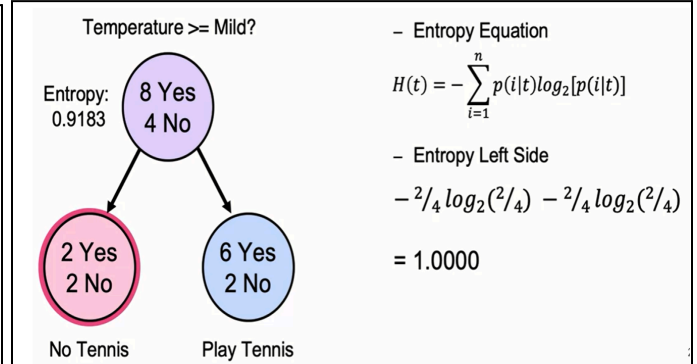
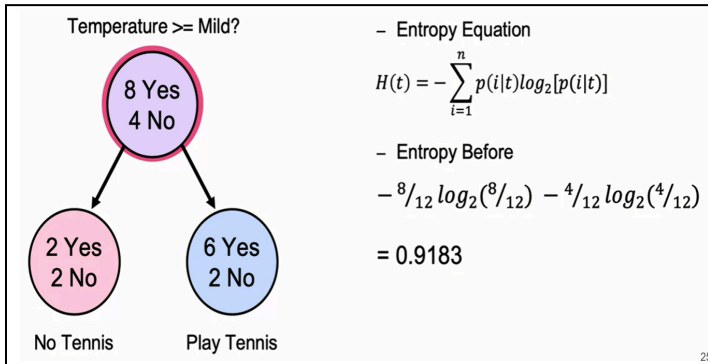
Predict whether people will play tennis based on outlook, temperature, humidity, and wind.

### **Example (regression):**

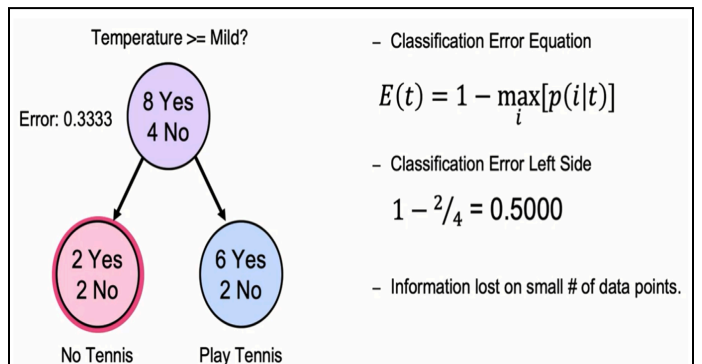
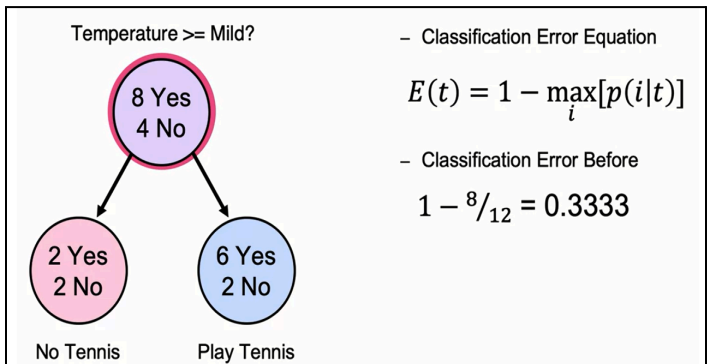
Predict average rainfall from slope and elevation.

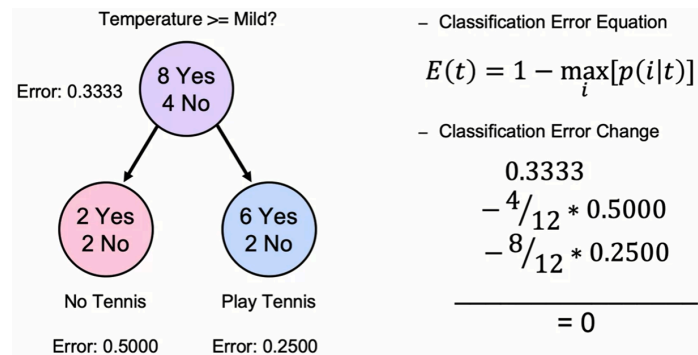
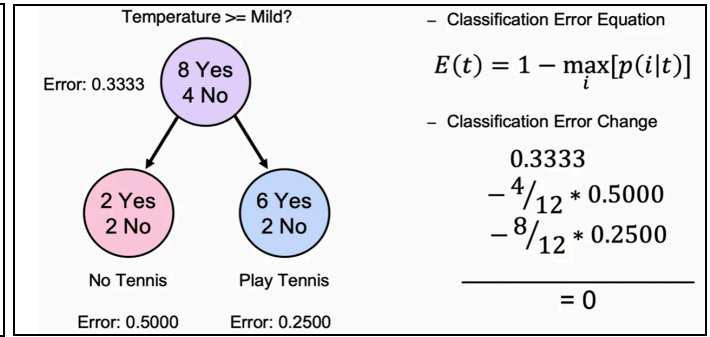
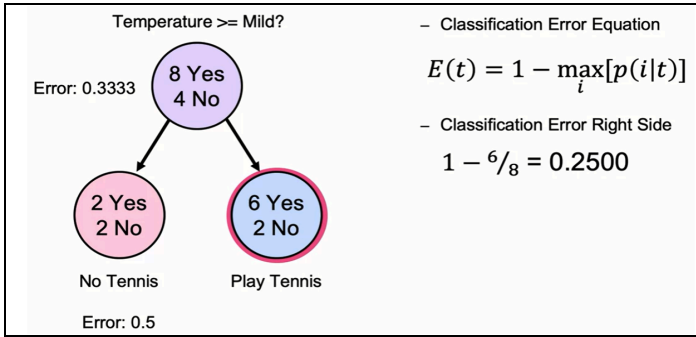
## Splitting Criteria

- The algorithm chooses the best split based on impurity reduction.
- Common impurity measures:
  - **Entropy:**

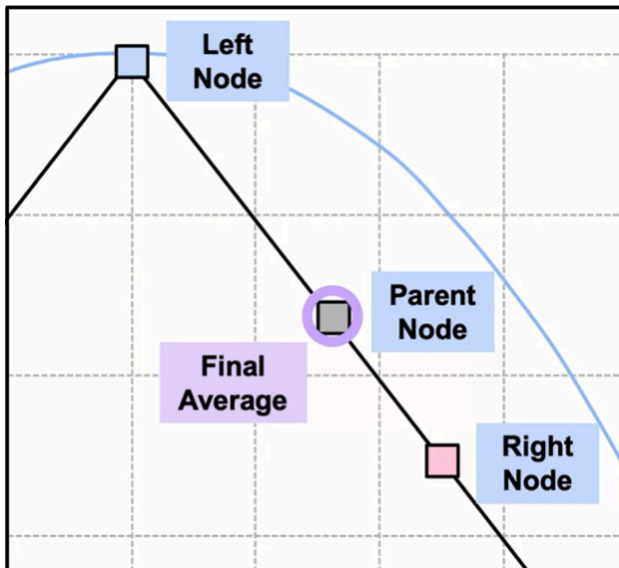


### ○ **Classification Error:**

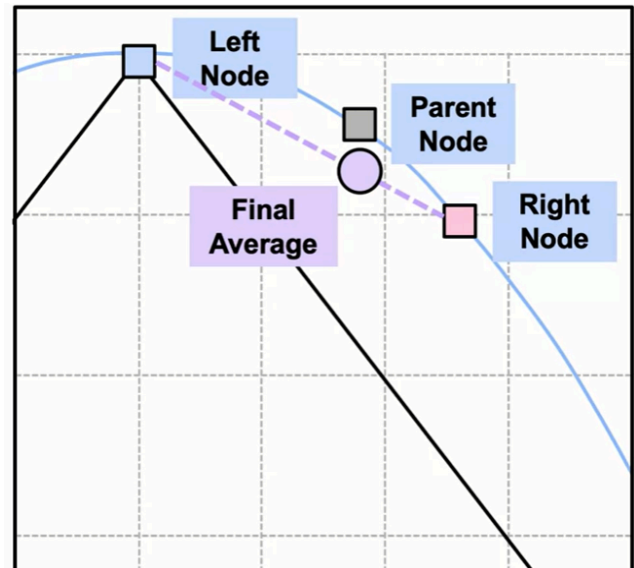




### ○ Information Gain by Splitting:



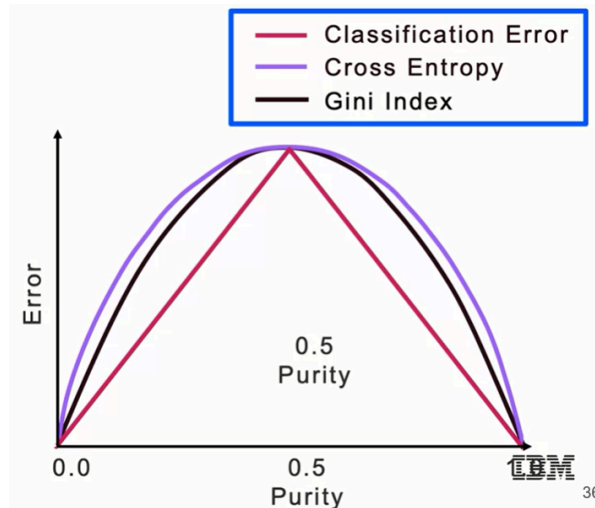
*Classification Error*



*Entropy*

- **Gini Index (Chỉ số Gini)**

$$G = 1 - \sum_i P(i|t)^2$$



- **Entropy** and **Gini** both encourage purer splits but **Gini** is simpler (no logarithm).

## Overfitting and Regularization

- **Decision Trees** easily overfit when allowed to grow deep — they fit the training set too well and generalize poorly.
- **Solutions:**
  - **Max Depth:** limit tree depth.
  - **Minimum Samples per Leaf:** require a minimum number of samples to split.
  - **Minimum Information Gain:** stop if improvement is too small.
  - **Pruning (Tỉa cây):** remove unnecessary branches after building the full tree.

## Pros and Cons

- **Advantages:**

- Easy to interpret and visualize.
- Works with mixed data (numeric/categorical).
- No need for scaling or normalization (chuẩn hóa dữ liệu).

- **Disadvantages:**

- High variance, prone to overfitting.
- Small data changes can lead to different splits.
- Less accurate alone—better when combined (e.g., Random Forest).

## Implementation in Python

```
from sklearn.tree import DecisionTreeClassifier

# Create and configure model
model = DecisionTreeClassifier(
    criterion='gini',    # or 'entropy'
    max_depth=5,
    min_samples_split=10)

# Train and predict
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

- To perform regression instead:

```
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=5)
```

## Comparison with Other Models

| Model                      | Fitting Speed       | Prediction Speed | Decision Boundary        | Notes                   |
|----------------------------|---------------------|------------------|--------------------------|-------------------------|
| <b>KNN</b>                 | Fast (No training)  | Slow             | Flexible (non-linear)    | Distance-based          |
| <b>Logistic Regression</b> | Slow (Optimization) | Fast             | Linear                   | Require feature scaling |
| <b>Decision Tree</b>       | Moderate            | Fast             | Non-linear, hierarchical | Prone to overfitting    |

## Key Takeaways

- Decision Trees split data using Entropy or Gini Index to reduce impurity.
- They are greedy algorithms (no lookahead optimization).
- Pruning and cross-validation help prevent overfitting.
- Easy to interpret and deploy — ideal for exploratory modeling.